

Content-Addressed Versioned Data Store: A Git-Inspired Approach to Structured Data Versioning Using Merkle Search Trees

[Student Name 1]¹, [Student Name 2]¹

¹Department of Computer Science and Engineering, [College/University Name]

[student1email]@[domain], [student2email]@[domain]

Guide: [Faculty Guide Name], [Designation]

Abstract

Saving versions of a dataset, comparing them, and rolling back to an earlier state is routine for source code because of Git, but it is still expensive to do properly for structured, high-volume data. Most existing approaches take one of two shortcuts: they copy the whole dataset for every saved version, which wastes storage, or they compare two versions record by record, which gets slow as the dataset grows. In this project we build a versioned data store on top of a content-addressed search tree, where every node in the tree is identified by a hash of what it contains. If two versions of the data share most of their content, they end up sharing most of their tree nodes too, so nothing gets duplicated, and finding what changed between two versions just means comparing hashes and only looking closer where they differ. This report covers the motivation for the project, the related work we looked at, the proposed architecture (split into a storage engine and a versioning layer), how we plan to evaluate it against a naive baseline, and where the implementation currently stands, including a small in-memory prototype that already shows the core hashing and diff mechanism working. We expect the finished system to show diff time and storage overhead that depend on how much data actually changed rather than on the total size of the dataset, while still supporting Git-style operations like commit, checkout, log, and branch on structured records.

Keywords: version control systems, Merkle search trees, content-addressed storage, database storage engines, diff algorithms

1 Introduction

Version control is one of the more taken-for-granted ideas in software engineering. Tools like Git let a developer save a snapshot of a codebase, look back through its history, compare any two points in that history, and undo a change if something goes wrong. The same kind of capability is now wanted for structured data as well, things like database tables, configuration state, ML feature sets, or datasets that several people edit at once, where teams need to know how the data evolved, who changed what, and how to safely revert a bad update. The catch is that structured data does not behave like source code. It is usually larger, changes more often, and is queried directly rather than just read as text, so simply borrowing Git's design does not work well in practice.

The core problem is that the two common ways of versioning data both have real drawbacks. The first, keeping a full copy per version, means storage cost keeps growing with every version you save, even if only a handful of rows actually changed. The second, comparing two versions by scanning through every record, means the time it takes to compute a diff grows with the size of the dataset, no matter how small the actual change was. Neither of these scales well once you have a dataset with many versions and a large number of records, which is the normal situation for any real system.

This matters for a few concrete reasons. Reproducible machine learning experiments need exact historical snapshots of the training data used. Regulated industries need an auditable, tamper-evident history of what changed and when. Collaborative or offline-first applications need a fast way to compare two states that may have diverged. A storage design where both storage cost and diff cost depend on how much actually changed, rather than on the size of the dataset, would help all three of these cases at once.

This project proposes a versioned data store built on a content-addressed search tree, where each node's identity is a hash of its own content, following the general idea used in production systems such as Dolt and originally introduced by the Noms database. Since two versions that share most of their data will also share most of their tree nodes, unchanged data never needs to be copied, and the difference between two versions can be found by comparing hashes from the root down and only descending where they disagree. Our contribution here is a from-

scratch implementation of this idea, split into a storage engine (pages, buffer pool, write-ahead logging, transactions) and a versioning layer built on top of it (commit graph, diff, checkout, branching), along with a direct benchmark of diff time and storage overhead against a naive baseline.

The rest of this report is organized as follows. Section 2 covers related work in version control, content-addressed storage, and conflict-free replicated data types, and points out the specific gap this project is trying to fill. Section 3 describes the proposed architecture, the core algorithms, the implementation approach, and how we plan to evaluate the system. Section 4 lays out the expected results and the evaluation plan. Section 5 concludes with the current status of the work and what is planned next.

2 Related Work

Version control and content-addressed storage have been studied both as production systems and as more formal distributed systems research. This section summarizes the prior work most relevant to our design and explains where our project fits relative to it.

Git, the version control system most developers already use, models a repository as a graph of commits, where each commit points to a tree of content-addressed blobs [1]. That content-addressing is what lets Git store unchanged files across commits without duplicating them, but Git's model is built around files, not structured records, so it doesn't give you efficient sub-record diffing or indexed queries, which limits how directly it can be applied to a database-style workload.

Auvolat and Taïani formalized the Merkle Search Tree, a search tree where each node's position is determined probabilistically from a hash of its key, so the resulting tree shape is deterministic and independent of insertion order [2]. Because of this, two independently built trees over similar data end up with structurally similar shapes, which the authors show makes the structure useful as a primitive for anti-entropy and state-based CRDT synchronization between replicas. We adapt the same idea of a content-addressed, hash-based tree, but apply it to single-writer, commit-based versioning rather than multi-replica convergence.

Dolt, a SQL database built specifically for version control, stores table and schema data using a related structure called a Prolly Tree: a content-addressed, probabilistically balanced B-Tree that was originally built for the Noms database project for exactly this purpose [4]. Dolt combines Prolly Trees with a Git-style commit graph, and its own documentation reports that this combination keeps both diff computation and storage overhead proportional to the size of the actual change rather than the size of the table, while also supporting branching at database scale [4]. Dolt is the closest existing system to what we are building, and we use its published architecture as a design reference rather than reusing any of its code. Our contribution is an independent implementation together with a controlled benchmark against a naive baseline, something that isn't really the focus of Dolt's own public documentation.

A related but different line of work is Conflict-free Replicated Data Types, formalized by Shapiro et al., which define data types whose replicas are guaranteed to converge to the same state no matter what order updates are delivered in [5]. CRDTs solve concurrent multi-writer conflict resolution, which is a harder problem than the single-writer versioning we are building. We treat CRDT-style automatic merging as explicitly out of scope for now, since correctly auto-resolving conflicts over arbitrary structured data is still an open research problem. Branching, meaning a named pointer to a version, is in scope; three-way merge with automatic conflict resolution is left as future work.

A different architectural approach is event sourcing, where a system stores the sequence of operations that produced the current state instead of the state itself, and rebuilds any historical state by replaying those operations [6]. This gives cheap diffing between adjacent versions, since two consecutive versions differ by exactly one stored operation, but comparing two versions that are far apart in history means replaying every operation in between, so the cost grows with how far apart the versions are rather than with how much actually changed. This is part of why our design is state-based rather than operation-based. We treat an operation log as a possible secondary structure for future comparison rather than the primary storage model.

None of this is useful at database scale without a proper storage engine underneath it: page-based storage, buffer-pool caching, and crash recovery through write-ahead logging. These are addressed by long-standing database

systems work, including the classical analysis of B-Tree indexing [8] and the ARIES write-ahead logging protocol for recoverable transactions [9]. Our storage engine follows these established techniques, adapted to support an immutable, content-addressed node model instead of the mutable in-place updates that classical B-Tree implementations assume.

Put together, prior work shows that content-addressed, hash-based trees are a proven way to get efficient diffing and structural sharing, that multi-writer conflict resolution is a much harder and largely open problem, and that the operation-based alternative has a real but different cost profile. What's missing is a documented, from-scratch, benchmarked implementation that isolates and directly measures the sub-linear diff and storage-sharing behavior that production systems like Dolt claim, including a controlled comparison against both a naive full-scan baseline and an operation-log baseline. That is the gap this project is aimed at.

3 Proposed Methodology

3.1 Overall Approach

The system is a key-value versioned store built on a content-addressed search tree. Every write updates the tree and produces a new root hash representing the full state at that point. Committing a version just means recording this root hash under a version identifier. Because tree nodes are identified by the hash of their own content, two versions that differ in only a small part of their data will share almost all of their tree nodes, and that sharing is what we use to avoid duplicating storage and to compute diffs quickly.

3.2 System Architecture

The system splits into two layers with a clean interface between them, which also maps directly onto how the two of us are dividing the work. The storage engine layer handles durability and low-level data management: page-based storage, a buffer pool with cache eviction, a write-ahead log for crash recovery, and transaction handling. The versioning layer sits on top of the storage engine's key-value interface and handles the version-control semantics: the content-addressed tree itself, the commit graph, the diff algorithm, checkout and rollback, and branching. Table 1 summarizes this split.

Table 1. Division of system responsibilities between team members.

Layer	Owner	Responsibilities
Storage engine	Partner A	Page manager, buffer pool (LRU eviction), write-ahead log, crash recovery, transactions
Versioning layer	Partner B	Content-addressed tree (nodes hashed by content), commit graph, diff algorithm, checkout/rollback, branching, benchmarking

The client talks to the system through a small command interface: commit, diff, log, checkout, branch. Every commit writes any new tree nodes to durable storage through the storage engine and records the resulting root hash in the commit graph; nodes that already exist under the same hash are simply referenced, not rewritten. A diff walks two trees in parallel from their roots, comparing node hashes level by level, and only recurses into subtrees where the hashes disagree, returning the actual set of keys that changed.

3.3 Algorithms and Techniques

- Content-addressed node hashing: each tree node's identifier is a cryptographic hash (SHA-256) of its own content, so structurally identical subtrees across any two versions collapse to the same identifier and are stored only once.
- Hash-guided diff: $\text{diff}(\text{rootA}, \text{rootB})$ compares node hashes recursively; when two nodes being compared share a hash, the algorithm stops there since the subtrees are provably identical, and only recurses into branches that don't match, so the cost of a diff tracks the amount of actual change rather than the total dataset size.
- Commit graph: each commit stores a pointer to its parent commit and its own root hash, the same directed-acyclic-graph model Git uses, which supports history traversal (log) and lays the groundwork for branch/merge support later.
- Write-ahead logging: mutations are appended to a log before being applied to the durable pages, following the ARIES-style logging discipline, so state can be rebuilt after a crash by replaying the log.

3.4 Data and Evaluation Inputs

We plan to evaluate the system using synthetically generated key-value datasets of varying size, roughly 1,000 to over 100,000 keys, with controlled mutation rates between versions, so diff time and storage overhead can be measured against both dataset size and how much of the data actually changed. We chose a synthetic setup on purpose so the relationship between change size and cost can be isolated and measured directly, rather than depending on the quirks of any one external dataset.

3.5 Implementation Details

The prototype is written in Python for now, since it lets us iterate quickly during early development, using the standard library's `hashlib` for content hashing. The storage engine pieces (buffer pool, WAL, page format) are being built as a self-contained module behind a narrow key-value interface, so the underlying language or storage backend can be revisited later without touching the versioning layer above it. Development follows a normal version-controlled workflow with automated tests covering tree construction and diff correctness.

3.6 Process Flow

- The client issues a write (put or delete) for one or more keys.
- The versioning layer updates the in-memory tree and recomputes node hashes bottom-up to the root.
- On commit, any tree nodes not already present in storage (by hash) get persisted through the storage engine; nodes that already exist are referenced, not duplicated.
- The new root hash is recorded in the commit graph with a pointer back to its parent commit.
- A diff request between two commits triggers a parallel hash-comparison walk of both trees, returning only the keys that actually changed.

3.7 Evaluation Setup

We use three metrics to compare the system against a naive baseline that stores a full copy per version and computes diffs by comparing every key one by one: diff time as a function of dataset size at a fixed change rate, diff time as a function of change size at a fixed dataset size, and storage overhead per commit, reported both as an absolute figure and as a percentage of nodes reused versus newly written. We also plan a secondary comparison against an operation-log baseline, to find the point at which each design becomes preferable, as discussed in Section 2.

4 Expected Results and Evaluation Plan

Since this report is being prepared for the first project review, before the full storage engine is built, this section describes our evaluation plan and what we expect the results to look like, rather than final measured numbers. We do already have a small in-memory prototype built ahead of this review to check that the core idea actually works: it builds a content-addressed tree, commits, applies a small mutation, commits again, and computes the diff between the two commits, printing out how many tree nodes were newly written versus reused from the previous version. Even at this small scale, the prototype shows that the hash-based structural sharing and hash-guided diff described in Section 3 behave the way we intended.

Based on the design in Section 3, we expect the finished system to show diff time that scales with the number of keys that actually changed between two versions, not with the total dataset size, unlike the naive baseline, where diff time should scale with total dataset size regardless of how much changed. Storage overhead per commit should stay small and roughly proportional to the size of the change, instead of growing linearly with the full dataset size the way a full-copy baseline does. We also expect the operation-log comparison to show that an operation log is cheaper for diffing adjacent commits, but that its cost grows with the number of commits in between when comparing versions that are far apart, while our tree's diff cost should stay largely independent of how far apart the commits are. These are expected trends rather than confirmed numbers, and testing them directly is exactly what the benchmark suite planned for Phase 3 is for.

Table 2. Project phases and milestones.

Phase	Milestone
Phase 1	Storage engine: page store, buffer pool, WAL (Partner A). Commit and diff data model design (Partner B).

Phase 2	Integration: content-addressed tree wired to durable pages and the commit graph.
Phase 3	Full benchmark suite: diff time and storage overhead against the naive and operation-log baselines.
Phase 4	Final report, results write-up, and demonstration.

5 Conclusion and Future Work

This report has laid out the design of a content-addressed versioned data store meant to provide Git-style versioning, commit, diff, checkout, and branching, over structured key-value data, with diff and storage cost that scale with the actual change rather than with dataset size. The design follows established ideas from Merkle Search Trees and content-addressed database engines like Dolt, but our contribution is an independent implementation plus a direct benchmark against both a naive baseline and an operation-log baseline, which is a comparison existing system documentation doesn't really provide in a controlled, reproducible way.

Once complete, the project's main contributions will be a working content-addressed storage engine with crash recovery through write-ahead logging, a versioning layer supporting commit, diff, checkout, and branching, and an empirical benchmark showing when this design actually beats the simpler alternatives. That matters for anything that needs efficient historical versioning of structured data: reproducible ML pipelines, auditable data systems, and collaborative or offline-first applications.

Beyond the current scope, planned future work includes three-way merge with conflict detection between branches, a rolling-hash chunking scheme for tree node boundaries so diffs stay stable under small edits (the same refinement Dolt made over the original Noms design), and testing on larger, real-world structured datasets in addition to the synthetic benchmark suite.

6 References

- [1] Chacon, S., Straub, B.: Pro Git, 2nd edn. Apress, New York (2014)
- [2] Auvolat, A., Taïani, F.: Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. In: 2019 38th IEEE International Symposium on Reliable Distributed Systems (SRDS), pp. 221-22109. IEEE (2019)
- [3] Merkle, R.C.: A Digital Signature Based on a Conventional Encryption Function. In: Advances in Cryptology, CRYPTO 1987, LNCS vol. 293, pp. 369-378. Springer (1988)
- [4] DoltHub: Prolly Trees, Storage Engine Architecture. <https://docs.dolthub.com/architecture/storage-engine> (accessed 2026)
- [5] Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M.: Conflict-free Replicated Data Types. In: Stabilization, Safety, and Security of Distributed Systems (SSS 2011), LNCS vol. 6976, pp. 386-400. Springer (2011)
- [6] Fowler, M.: Event Sourcing. [martinfowler.com. https://martinfowler.com/eaDev/EventSourcing.html](http://martinfowler.com/eaDev/EventSourcing.html) (accessed 2026)
- [7] Boodman, A., et al.: Noms, The Decentralized Database. <https://github.com/attic-labs/noms> (accessed 2026)
- [8] Comer, D.: The Ubiquitous B-Tree. ACM Computing Surveys 11(2), pp. 121-137 (1979)
- [9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., Schwarz, P.: ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. ACM Transactions on Database Systems 17(1), pp. 94-162 (1992)